

(Numbers, Array, Conditions)

Quick Review on session 1

1. JavaScript Output Methods

- `document.write()`
- `console.log()` debugging
- `window.alert()`

2. JavaScript: Compiler و Interpreter?

JavaScript **Interpreted language**

لكن engines الحديثة (زي V8) بتستخدم **JIT Compilation**

3. *var* vs *let* vs *const*

Keyword	Redeclare	Update	Scope
var	✓	✓	Function
let	X	✓	Block
const	X	X	Block

Block Scope (ES6):

```
if (true) {  
  let x = 10;  
}  
console.log(x); // Error
```

Key idea:

- `var` ignores blocks
- `let` & `const` respect blocks `{ }`

- يعني ببساطه **var** مبهتمش غير بالاقواس بتاعت ال **function** بس انما اي اقواس تانيه مبهتمش بيها ولما نعرفها جوا الاقواس لما نيجي نستدعيها برا الاقواس مش هيدينا ايرور
 - ودا ال مش موجود في **let and const** لاننا لو عرفناهم جوا اي **block** او اقواس لما نيجي نستدعيهم برا **هيطلعوا ايرور**
- يعني ES6 هي جديدة لغة مش Version نفس من جديدة JavaScript ضافت ل features جديده

ES6 = ECMAScript 2015

- *Standard for JavaScript / Added modern features made JS closer to Backend languages*
- features ضافت JavaScript جديدة من نفس **Version** مش لغة جديدة هي ES6 يعني **Modules** و **Classes** و **Arrow Functions** و **const** و **let** زي JavaScript جديده ل **block scope** و

4. Automatic Semicolon Insertion (ASI)

بتضيف ; بشكل تلقائي

5. Data Types

Type	Mutable / Immutable	Copying method	Examples
Primitive	Immutable	Copy by value	string, Numbers, Boolean, null, undefined
Non-Primitive	Mutable	Copy by reference	object, array, function

Mnemonic:

"Primitive → cannot change → create new copy"

"Non-Primitive → can change → affects all reference"

```
typeof "John" // "string"
typeof null   // "object" ! JS bug`

//Number=immutable
let a = 10;
```

```
let b = a;
b = 20;
a // 10

//Object / Array = Mutable
let o1 = {name: "Sara"};
let o2 = o1;
o2.name = "Lara";
o1.name // "Lara"
```

6. null vs undefined

Type	Meaning	Type
undefined	JS assigned	undefined
null	Developer assigned	object(bug)

```
null == undefined // true
null === undefined // false
```

7. Operators

- Comparison:

```
5 == "5" // true
5 === "5" // false
```

- Logical:

```
true && false // false
```

- Increment and Decrement:

```
x++; x--;
```

8. NaN

- NaN = Invalid Number
- NaN $\neq$ **itself**

```
5/"apc" -> NaN
```

```
NaN === NaN // false
```

```
//Check using:  
isNaN(value)
```

- Session 2

Numbers:

1- JS Numbers:

- JavaScript numbers are always double (*64-bit floating point*)

- يعني في لغات تانيه ال number بيتقسم لانواع سواء int او float او long او double وبيختلف ال size بتاعهم ولكن هنا هو 64 لكه ودا بيسبب *issue* بسبب ال floating point ان الفاصله مكانها مش ثابت يعني بتتحرك وبتظهر مع الارقام العشريه (*دي مش bug في JavaScript*)

```
console.log(0.1+0.2);  
  
//to fix it  
Number((0.1+0.2).toFixed(2))  
  
(0.2+0.1).toFixed(2)===0.3
```

- واحده من ال *methods* هي `toFixed` دي لما منحطش جواها اي رقم هتعمل *round* للرقم يعني هتقربه لاقرب رقم صحيح انما لو حطينا رقم ف كدا انا بقولها اعرضلي كام رقم من بعد العلامة.
Note -> `toFixed` بترجع ف الاصل الرقم علي هيئه استرينج فانا بستخدم نامبر عشان احوله لنامبر

- Difference between Number and object Number

```
let x = 5; //primitive
let y = new Number(5); //non primitive

console.log(x == y);
console.log(x === y);
```

-> *Auto Boxing*:

دي حاجه بتحصل جوا js ان عشان الرقم العادي يقدر يستخدم ال methods ال موجوده جوا ال Number object ف js بتحوله ل object وتستخدم ال method بعدين يرجع ثاني ل number عادي

```
let x = 5;

x.toString();
console.log(typeof (x)); //string

console.log(typeof (x.toString())); //number
```

2- JS Number Methods

- `toString()` : Converts number to a string.
- `toFixed(x)` : Formats number with fixed number of decimals
- `isNaN(x)` : بتسأل: هل القيمة دي ينفج تتحول لرقم؟
- `Number.isNaN()` : بتسأل: هل القيمة دي هي فعلا نان ذات نفسها ولا لا:
- `parseInt(x)` : Converts string to integer (stops at first non-digit).
- `Number.isInteger(x)` : Checks if value is an integer.

```
let x = 123;
x.toString();

let x = 10.5678;
x.toFixed(2);

isNaN("abc");
Number.isNaN("abc");

parseInt("10");

Number.isInteger(10);
```

-> Note:

في حاجتين عشان نحول من string ل Number هما (parseInt , Number) ولكن في فرق بينهم

- **اولا Number :**

هي strict شوي لانها بتحول الاسترينج ال لازم يكون كله ارقام لو فيه حروف هيرجع NaN وبتشيل المسافات ال موجوده ف الاول (+ هي اختصار لل Number)

- **ثانيا parseInt :**

هي flexible لان لو الاسترينج جوا حرف مثلا هيوقف عند الحرف ويحول الباقي لرقم وهي كمان لو فيه كسور بترجع الرقم صحيح

```
Number("10"); // 10
Number("10.5"); // 10.5
Number("10px"); // NaN
Number("abc"); // NaN
Number(" 10 "); // 10

parseInt("10"); // 10
parseInt("10.5"); // 10
parseInt("10px"); // 10
parseInt("px10"); // NaN
parseInt(" 10 "); // 10
```

• لو الميثود بتسأل **سؤال عن القيمة**

Static(**Number.isInteger**) →

لو بتعمل **تعديل / تنسيق للقيمة** فهنا محتاج اعمل ال auto boxing

→ Instance(**toFixed()**)

3- JS Number Properties

معلومة (قيمة مخزنة جوه اوبجيكيت بتوصفه أو بتقول معلومة عنه) **Property =**

Method = فعل

Property	Description (English)
Number.MAX_VALUE	The largest possible number in JavaScript
Number.MIN_VALUE	The smallest positive number greater than 0
Number.POSITIVE_INFINITY	Represents positive infinity
Number.NEGATIVE_INFINITY	Represents negative infinity

4- JS Bitwise Operators

يعمل logical operators علي الارقام يعني بحولهم لباينري الاول

5 & 1

Conditions

```
if (condition) {  
  // true  
} else {  
  // false  
}  
  
//example  
let grade = 85;  
  
if (grade >= 90) {  
  console.log("Excellent");  
} else if (grade >= 75) {  
  console.log("Very Good");  
} else if (grade >= 50) {  
  console.log("Pass");  
} else {  
  console.log("Fail");  
}  
  
// logical operators الكونديشان ممكن يكون فيه  
if(grade>50 && grade<60)
```

-> *JS Ternary Operator*:

دي نوع من انواع ال operators بيختصر شكل if else في سطر واحد لما الشكل يكون بسيط
وكمان ف ال assigning values

```
condition ? valueIfTrue : valueIfFalse;  
  
let age = 18;  
let result = (age >= 18) ? "Adult" : "Minor";
```

-> **JS Switch**:

هي بديل ل if else الكثيره ولكن لو ال condition فيه equal انما لو هنستخدم اي حاجه ثانيه زي (< , > , =< , =>) مينفعش نستخدم switch

```
let day = 3;

switch (day) {
  case 1:
    console.log("Saturday");
    break;
  case 2:
    console.log("Sunday");
    break;
  case 3:
    console.log("Monday");
    break;
  default:
    console.log("Unknown day");
}
```

How to make user to enter a number:

```
let num= window.prompt();
console.log(0.025*num);
```

هنا انا بخلي اليوزر يدخل رقم ويتخط فالتغير واعمل انا حساباتي

Arrays

Heterogeneous / بيثيل أكثر من قيمة في متغير واحد **Array = container**

```
let arr=["ahmed","mona","ali",1,true];
console.log(arr);
console.log(arr.length);
arr[0]="Tahany";
console.log(arr);
const cars = new Array("Saab", "Volvo", "BMW");
```

-> Array في JS **Mutable** يعني عادي اعدل علي اي قيمه فيها :

-> **In JavaScript, Array is a special type of Object:**

- يعني كل array هي object ولكن مش كل object يعتبر array
- طب هي ليه object ؟ لان ليها attributes زي length و methods زي push مثلا

```
console.log(typeof arr);

// عشان افرق بين الاراي والابجكت
let x = {};
let y = [];

typeof x; // "object"
typeof y; // "object"

Array.isArray(x); // false
Array.isArray(y); // true
```

JS Array Methods:

-> *Array Add & Remove Methods*

Method	What it does (English)
<code>push()</code>	Adds elements to the end of the array
<code>pop()</code>	Removes the last element from the array
<code>unshift()</code>	Adds elements to the beginning of the array
<code>shift()</code>	Removes the first element from the array
<code>splice()</code>	Adds or removes elements at a specific index

- *splice*:

الأصلي **array** ويتغير **ال** array تُستخدم لـ **الإضافة / الحذف / التعديل** داخل **ال**

```
array.splice(start, deleteCount, item1, item2, ...)
```

Parameter	Meaning
<code>start</code>	اللي نبدأ منه Index
<code>deleteCount</code>	عدد العناصر اللي هتتحذف
<code>item</code>	عناصر جديدة تضاف (اختياري)

```
//only delete
let arr = ["a", "b", "c", "d"];
arr.splice(1, 2);
console.log(arr); // ["a", "d"]

//only addition
let arr = ["a", "d"];
arr.splice(1, 0, "b", "c");
console.log(arr); // ["a", "b", "c", "d"]

//delete and add
let arr = ["a", "b", "x", "d"];
arr.splice(2, 1, "c");
console.log(arr); // ["a", "b", "c", "d"]
```

Note: return value -> سيكون اراي بالعناصر ال اتحذفت ولو محذفتش هيرجع اراي فاضيه
زي shift and push

->Array Modify / Copy Methods

Method	What it does (English)
splice()	Modifies the array by adding/removing elements
slice()	Returns a copy of part of the array
concat()	Combines arrays into a new array
fill()	Replaces all elements with a static value

- **Slice**:
مش بتغير الأصل يعني بتعمل اراي جديده array من جزء من ال (copy) بتاخذ نسخة

ولكن احنا هنا مبنحسبش النهايه معانا ف العدد

```
array.slice(start, end)

let arr = ["a", "b", "c", "d"];
let part = arr.slice(1, 3);

console.log(part); // ["b", "c"]
console.log(arr); // ["a", "b", "c", "d"]
console.log(arr.slice()); // هي هنا هتطبع كل الاري
```

- **concat**
جديده و مش بتغير الأصل Array وتعمل (أو أكثر) Array مع Array بتدمج

```

let arr1 = [1, 2];
let arr2 = [3, 4];

let result = arr1.concat(arr2);

console.log(result); // [1, 2, 3, 4]
console.log(arr1);   // [1, 2]

let newArr = arr.concat(3, 4); // هنا بضيف قيم
let all = a.concat(b, c); // هنا بضيف اكثر من اراي

//difference between concat with + and concat
names1=['gamal', 'amged'];
names+=names1;
console.log(typeof names); // كذا هيطلعلي نوعها استرينج وانا عاوزه اراي
console.log(names.concat(names1)); // انا كذا دمجت الاتنين اراي مع بعض وهما
لسه اراي بردو زي م هما

```

- **fill** :
الأصلي Array بقيمة واحدة , بتغير ال Array بتملا ال

```

let arr = [1, 2, 3, 4];
arr.fill(0);
console.log(arr); // [0, 0, 0, 0]

//slice بمللي جزء من ال اراي وهنا النهايه مش معنا بردو زي
let arr = [1, 2, 3, 4, 5];
arr.fill(9, 1, 4);
console.log(arr); // [1, 9, 9, 9, 5]

```

-> Array Search Methods

Method	What it does (English)
<code>includes()</code>	Checks if a value exists in the array
<code>indexOf()</code>	Returns the first index of a value
<code>lastIndexOf()</code>	Returns the last index of a value

```

console.log(names.indexOf('ahmed', (يمكن احط بدايه البحث بتاعي, بترجع //
الانديكس
console.log(fames.lastIndexOf("ali", (بدايه البحث, //بشغل من ورا

```

وبترجع اندكس برودو
console.log(names.includes("ahmed", بتاعلي برودو, (ممکن احط بدايه البحث بتاعلي برودو, بترجع));
ترو او فولس

-> Array Order & Transform Methods

Method	What it does (English)
sort()	Sorts the array elements
reverse()	Reverses the array order
join()	Converts the array to a string
flat()	Flattens nested arrays

```
//sort and reverse
console.log(names.reverse());
console.log(names.sort());

//join
console.log(names.join()); //هي هنا هتعرضلي الاراي من غير الاقواس بتاعتها يعني
كانها ع هيئه استرينج
console.log(names.join('-')); //انا هنا ممكن احط اي حاجه بينهم زي اسبيس او
شرطه

//flat
let all=[[1,2],[3,4],[4,5]];
console.log(all.flat());
```

Note:

عندنا اثنين method بيحولوا ال array ل string هما (join / toString)

- ال toString : ال separator ثابت ال هي الفاصله (,) مينفعش اغيرها
- انما ف join : انا ممكن وانا بحول الاراي لاسترينج افصل بينهم ب اي حاجه سواء (/ او - او ,)

JS Array Reference

- أي تعديل -> يؤثر على الكل Array = Reference
- mutable يعني Non-Primitive لانها من النوع
- بيشاوروا على نفس المكان في الذاكرة a و b

```
let a = [1, 2];
let b = a;

b.push(3);
console.log(a); // [1,2,3]
```

solution :

-> **Spread Operator (...)**:

اتضافت في **ES6** بتفرد (spread) عناصر ال array واحدة واحدة وتعمل **نسخة جديدة** منها عشان
تحل مشكله ال reference ف ال array

```
let b = [ ... a];

b.push(4);

console.log(a); // [1,2,3]
console.log(b); // [1,2,3,4]
```

Notes (مهمة)

- **Mutates array**: `push` , `pop` , `shift` , `unshift` , `splice` , `sort` , `reverse` , `fill` , يعني بتغير ف نفس الاري الاصليه
 - **Returns new array**: `map` , `filter` , `slice` , `concat` , `flat` بتعمل اري جديده ومبتاثرش علي
-